

8.3-Polynomial_Regression

July 24, 2025

0.0.1 Polynomial Regression Implementation

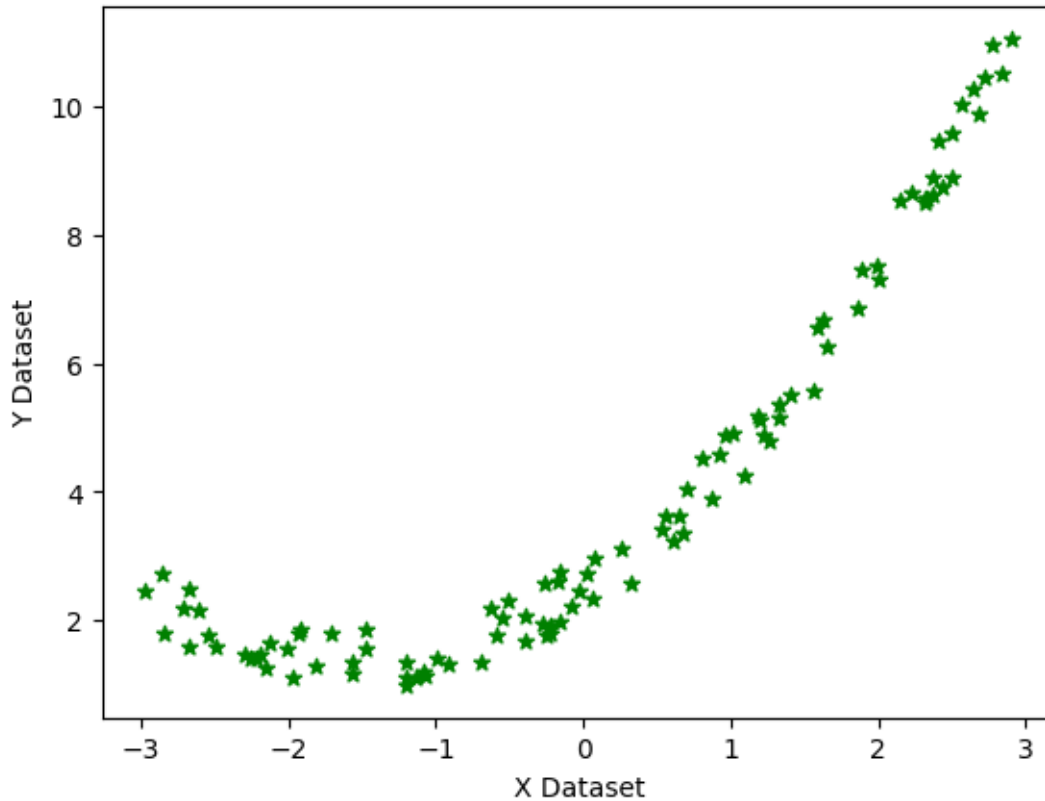
```
[1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import warnings
warnings.filterwarnings('ignore')
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
```

```
[2]: X = 6 * np.random.rand(100, 1) - 3
y = 0.5 * X**2 + 1.5*X + 2 + np.random.rand(100, 1)

## Quadratic Equation used =  $y = 0.5x^2 + 1.5x + \text{outliers}$ 
```

```
[3]: plt.scatter(X,y, c='g', marker="*")
plt.xlabel('X Dataset')
plt.ylabel("Y Dataset")
```

```
[3]: Text(0, 0.5, 'Y Dataset')
```



```
[4]: X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
↳ random_state=42)
```

```
[5]: regression_1 = LinearRegression()
regression_1.fit(X_train, y_train)
```

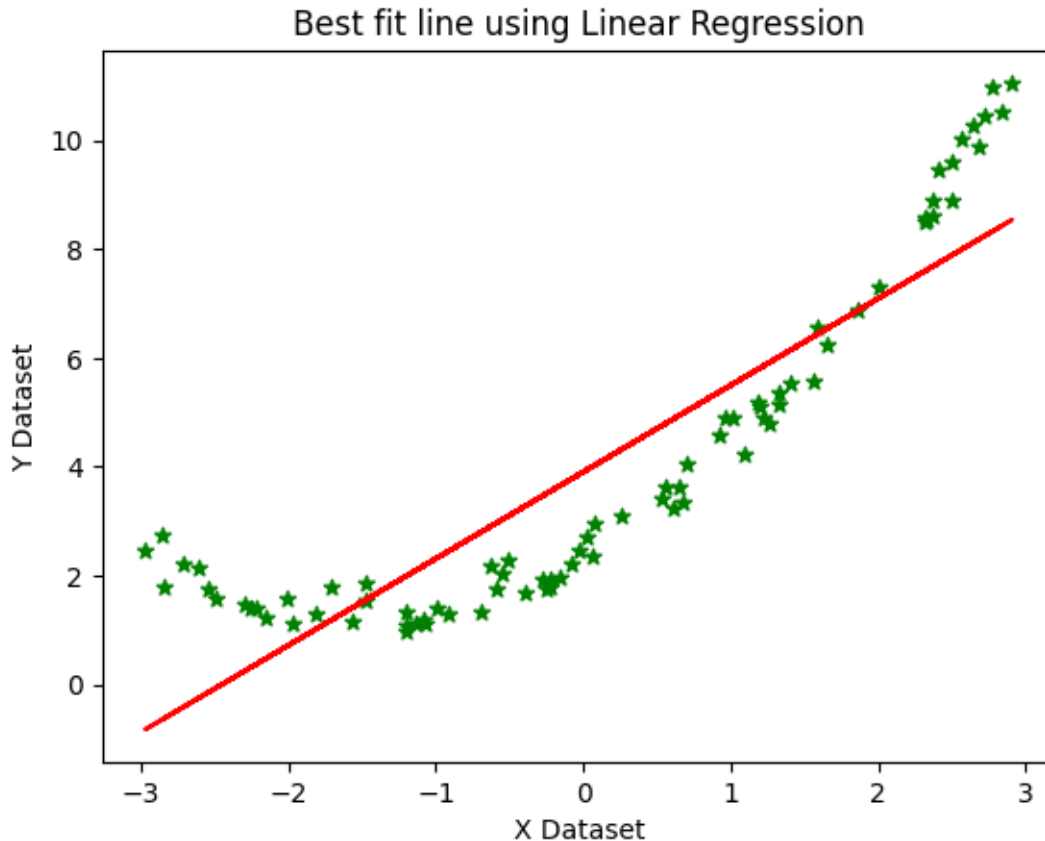
```
[5]: LinearRegression()
```

```
[6]: linear_score = r2_score(y_test, regression_1.predict(X_test))
linear_score
```

```
[6]: 0.7887808396222011
```

Plotting the Best fit line with respect to the Data distribution where the error is too much

```
[7]: plt.title("Best fit line using Linear Regression")
plt.plot(X_train, regression_1.predict(X_train), c = 'r')
plt.scatter(X_train, y_train, c='g', marker='*')
plt.xlabel("X Dataset")
plt.ylabel("Y Dataset")
plt.show()
plt.close()
```



0.0.2 Solving this problem using the Polynomial Transformation

```
[8]: from sklearn.preprocessing import PolynomialFeatures
```

```
[9]: poly = PolynomialFeatures(degree=2, include_bias=True) # It is add the 1 in
    ↪ equation like (y = b0*1 + b1x1 + b2x1^2 ) so here 1 is your include_bias =
    ↪ True
```

```
[10]: X_train_poly = poly.fit_transform(X_train)
    X_test_poly = poly.transform(X_test)
```

```
[11]: X_train_poly # Here the 1st value is 1, 2nd value is x1 and 3rd value is x^2
```

```
[11]: array([[ 1.00000000e+00, -2.25568511e+00,  5.08811530e+00],
 [ 1.00000000e+00,  1.19335789e+00,  1.42410306e+00],
 [ 1.00000000e+00,  2.56673485e+00,  6.58812781e+00],
 [ 1.00000000e+00,  1.86169314e+00,  3.46590136e+00],
 [ 1.00000000e+00, -2.75708189e-01,  7.60150053e-02],
 [ 1.00000000e+00, -1.97169153e+00,  3.88756747e+00],
 [ 1.00000000e+00, -1.20534222e+00,  1.45284987e+00],
```

```

[ 1.00000000e+00, 1.55655796e+00, 2.42287268e+00],
[ 1.00000000e+00, -1.20016389e+00, 1.44039337e+00],
[ 1.00000000e+00, 1.22346131e+00, 1.49685759e+00],
[ 1.00000000e+00, -2.18200504e-01, 4.76114601e-02],
[ 1.00000000e+00, 1.32580138e+00, 1.75774929e+00],
[ 1.00000000e+00, -2.21830575e+00, 4.92088041e+00],
[ 1.00000000e+00, 1.01200918e+00, 1.02416258e+00],
[ 1.00000000e+00, 1.41140568e+00, 1.99206601e+00],
[ 1.00000000e+00, 6.51479684e-01, 4.24425779e-01],
[ 1.00000000e+00, 1.08751008e+00, 1.18267818e+00],
[ 1.00000000e+00, 7.03305932e-01, 4.94639234e-01],
[ 1.00000000e+00, 2.71662253e+00, 7.38003796e+00],
[ 1.00000000e+00, 6.35468511e-02, 4.03820229e-03],
[ 1.00000000e+00, 2.64319658e+00, 6.98648816e+00],
[ 1.00000000e+00, 2.90912897e+00, 8.46303138e+00],
[ 1.00000000e+00, 9.63286587e-01, 9.27921048e-01],
[ 1.00000000e+00, -1.70514762e+00, 2.90752841e+00],
[ 1.00000000e+00, 2.40572790e+00, 5.78752671e+00],
[ 1.00000000e+00, -2.71490384e+00, 7.37070288e+00],
[ 1.00000000e+00, -2.83990964e+00, 8.06508677e+00],
[ 1.00000000e+00, 2.31376561e+00, 5.35351131e+00],
[ 1.00000000e+00, -2.25644803e-01, 5.09155769e-02],
[ 1.00000000e+00, 2.68474946e+00, 7.20787967e+00],
[ 1.00000000e+00, -2.54566474e-01, 6.48040897e-02],
[ 1.00000000e+00, 2.49729116e+00, 6.23646316e+00],
[ 1.00000000e+00, 2.49424278e+00, 6.22124706e+00],
[ 1.00000000e+00, 2.56215366e-01, 6.56463136e-02],
[ 1.00000000e+00, 1.58397282e+00, 2.50896990e+00],
[ 1.00000000e+00, 2.00157594e+00, 4.00630626e+00],
[ 1.00000000e+00, -6.88691216e-01, 4.74295591e-01],
[ 1.00000000e+00, 9.19413500e-01, 8.45321185e-01],
[ 1.00000000e+00, 2.36883889e+00, 5.61139768e+00],
[ 1.00000000e+00, -1.60921519e-01, 2.58957354e-02],
[ 1.00000000e+00, 1.17964304e+00, 1.39155771e+00],
[ 1.00000000e+00, -1.06847378e+00, 1.14163621e+00],
[ 1.00000000e+00, 1.32503642e+00, 1.75572152e+00],
[ 1.00000000e+00, -1.08045846e+00, 1.16739048e+00],
[ 1.00000000e+00, 5.58434058e-01, 3.11848597e-01],
[ 1.00000000e+00, 2.13412911e-02, 4.55450706e-04],
[ 1.00000000e+00, 5.30000537e-01, 2.80900569e-01],
[ 1.00000000e+00, -2.53509234e+00, 6.42669316e+00],
[ 1.00000000e+00, -9.92732464e-01, 9.85517745e-01],
[ 1.00000000e+00, -1.13058641e+00, 1.27822562e+00],
[ 1.00000000e+00, 2.83613681e+00, 8.04367202e+00],
[ 1.00000000e+00, -5.84090735e-01, 3.41161987e-01],
[ 1.00000000e+00, -2.00148279e+00, 4.00593337e+00],
[ 1.00000000e+00, -5.11457562e-01, 2.61588838e-01],

```

```
[ 1.00000000e+00,  2.77567735e+00,  7.70438473e+00],
[ 1.00000000e+00, -1.81225366e+00,  3.28426331e+00],
[ 1.00000000e+00, -1.47745656e+00,  2.18287788e+00],
[ 1.00000000e+00, -8.07791546e-02,  6.52527182e-03],
[ 1.00000000e+00,  6.70968142e-01,  4.50198248e-01],
[ 1.00000000e+00, -1.47843990e+00,  2.18578455e+00],
[ 1.00000000e+00, -5.49128128e-01,  3.01541701e-01],
[ 1.00000000e+00,  6.17469931e-01,  3.81269116e-01],
[ 1.00000000e+00, -1.20556444e+00,  1.45338562e+00],
[ 1.00000000e+00, -9.14697968e-01,  8.36672373e-01],
[ 1.00000000e+00,  1.26339490e+00,  1.59616667e+00],
[ 1.00000000e+00, -6.32047960e-01,  3.99484624e-01],
[ 1.00000000e+00,  2.37580691e+00,  5.64445848e+00],
[ 1.00000000e+00,  2.31306273e+00,  5.35025919e+00],
[ 1.00000000e+00, -2.14689898e+00,  4.60917522e+00],
[ 1.00000000e+00,  8.08960800e-02,  6.54417576e-03],
[ 1.00000000e+00, -2.85342893e+00,  8.14205666e+00],
[ 1.00000000e+00, -3.97188635e-01,  1.57758812e-01],
[ 1.00000000e+00, -2.96563010e+00,  8.79496190e+00],
[ 1.00000000e+00, -2.48839749e+00,  6.19212206e+00],
[ 1.00000000e+00, -1.56851922e+00,  2.46025254e+00],
[ 1.00000000e+00,  2.32572192e+00,  5.40898244e+00],
[ 1.00000000e+00, -2.29038346e+00,  5.24585638e+00],
[ 1.00000000e+00, -3.32739319e-02,  1.10715455e-03],
[ 1.00000000e+00, -2.60814290e+00,  6.80240941e+00],
[ 1.00000000e+00,  1.64856508e+00,  2.71776681e+00]]])
```

```
[12]: X_test_poly
```

```
[12]: array([[ 1.          , -1.56952266,  2.46340137],
 [ 1.          ,  2.22199062,  4.9372423 ],
 [ 1.          , -1.92685479,  3.71276937],
 [ 1.          , -1.92182154,  3.69339802],
 [ 1.          , -0.39535683,  0.15630702],
 [ 1.          ,  2.14621368,  4.60623316],
 [ 1.          , -2.12555263,  4.51797397],
 [ 1.          ,  1.62651926,  2.64556491],
 [ 1.          , -2.67330705,  7.14657058],
 [ 1.          ,  0.33024547,  0.10906207],
 [ 1.          ,  0.8673569 ,  0.752308  ],
 [ 1.          ,  2.43829124,  5.94526415],
 [ 1.          ,  1.98604914,  3.94439119],
 [ 1.          , -2.67001488,  7.12897946],
 [ 1.          , -0.25590037,  0.065485  ],
 [ 1.          ,  0.80358221,  0.64574437],
 [ 1.          , -0.15205104,  0.02311952],
 [ 1.          ,  1.89019652,  3.57284288],
```

```
[ 1.          , -2.19466758,  4.8165658 ],
[ 1.          , -0.16823549,  0.02830318]])
```

Let's Analyze the r2 score

```
[13]: regression_2 = LinearRegression()
      regression_2.fit(X_train_poly, y_train)
      y_pred = regression_2.predict(X_test_poly)
      polynomial_score = r2_score(y_test, y_pred)
      polynomial_score
```

```
[13]: 0.9792573702598509
```

```
[14]: print(f"r2_score with Linear Regression: {linear_score} \nr2_score with_\n
      ↪Polynomial Regression: {polynomial_score}")
```

```
r2_score with Linear Regression: 0.7887808396222011
r2_score with Polynomial Regression: 0.9792573702598509
```

```
[15]: regression_1.coef_
```

```
[15]: array([[1.59463062]])
```

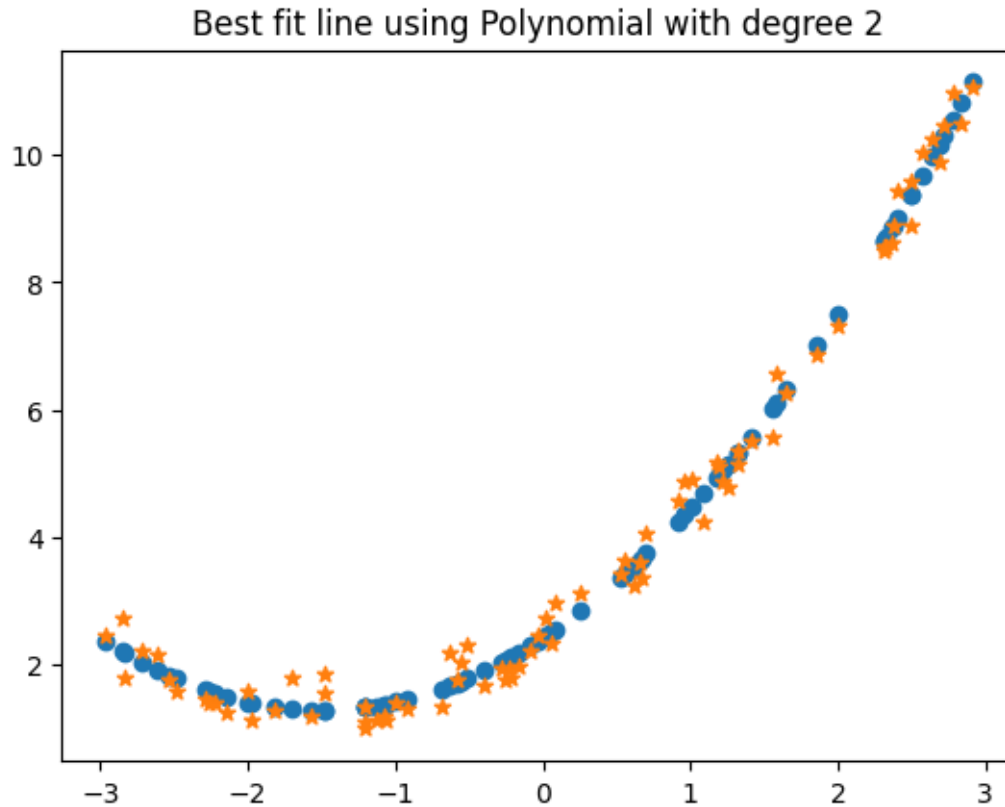
```
[16]: regression_2.coef_
```

```
[16]: array([[0.          ,  1.52231799,  0.50783152]])
```

0.0.3 Visualize the Polynomial Regression Best fit line

```
[17]: plt.title("Best fit line using Polynomial with degree 2")
      plt.scatter(X_train, regression_2.predict(X_train_poly))
      plt.scatter(X_train, y_train , marker='*')
```

```
[17]: <matplotlib.collections.PathCollection at 0x28988308d00>
```



0.0.4 Let's apply the deg=3

```
[18]: poly = PolynomialFeatures(degree=3, include_bias=True) # It is add the 1 in
      ↪ equation like (y = b0*1 + b1x1 + b2x1^2 + b3x1^3) so here 1 is your
      ↪ include_bias = True
      X_train_poly = poly.fit_transform(X_train)
      X_test_poly = poly.transform(X_test)
```

```
[19]: X_train_poly ## here you see the 4 points which is b0, b1, b2 and b3
```

```
[19]: array([[ 1.00000000e+00, -2.25568511e+00,  5.08811530e+00,
               -1.14771859e+01],
              [ 1.00000000e+00,  1.19335789e+00,  1.42410306e+00,
                1.69946462e+00],
              [ 1.00000000e+00,  2.56673485e+00,  6.58812781e+00,
                1.69099773e+01],
              [ 1.00000000e+00,  1.86169314e+00,  3.46590136e+00,
                6.45244479e+00],
              [ 1.00000000e+00, -2.75708189e-01,  7.60150053e-02,
               -2.09579594e-02],
              [ 1.00000000e+00, -1.97169153e+00,  3.88756747e+00,
```

```

-7.66508384e+00],
[ 1.00000000e+00, -1.20534222e+00, 1.45284987e+00,
-1.75118129e+00],
[ 1.00000000e+00, 1.55655796e+00, 2.42287268e+00,
3.77134174e+00],
[ 1.00000000e+00, -1.20016389e+00, 1.44039337e+00,
-1.72870812e+00],
[ 1.00000000e+00, 1.22346131e+00, 1.49685759e+00,
1.83134735e+00],
[ 1.00000000e+00, -2.18200504e-01, 4.76114601e-02,
-1.03888446e-02],
[ 1.00000000e+00, 1.32580138e+00, 1.75774929e+00,
2.33042642e+00],
[ 1.00000000e+00, -2.21830575e+00, 4.92088041e+00,
-1.09160173e+01],
[ 1.00000000e+00, 1.01200918e+00, 1.02416258e+00,
1.03646194e+00],
[ 1.00000000e+00, 1.41140568e+00, 1.99206601e+00,
2.81161329e+00],
[ 1.00000000e+00, 6.51479684e-01, 4.24425779e-01,
2.76504773e-01],
[ 1.00000000e+00, 1.08751008e+00, 1.18267818e+00,
1.28617444e+00],
[ 1.00000000e+00, 7.03305932e-01, 4.94639234e-01,
3.47882708e-01],
[ 1.00000000e+00, 2.71662253e+00, 7.38003796e+00,
2.00487774e+01],
[ 1.00000000e+00, 6.35468511e-02, 4.03820229e-03,
2.56615040e-04],
[ 1.00000000e+00, 2.64319658e+00, 6.98648816e+00,
1.84666616e+01],
[ 1.00000000e+00, 2.90912897e+00, 8.46303138e+00,
2.46200498e+01],
[ 1.00000000e+00, 9.63286587e-01, 9.27921048e-01,
8.93853899e-01],
[ 1.00000000e+00, -1.70514762e+00, 2.90752841e+00,
-4.95776515e+00],
[ 1.00000000e+00, 2.40572790e+00, 5.78752671e+00,
1.39232145e+01],
[ 1.00000000e+00, -2.71490384e+00, 7.37070288e+00,
-2.00107496e+01],
[ 1.00000000e+00, -2.83990964e+00, 8.06508677e+00,
-2.29041177e+01],
[ 1.00000000e+00, 2.31376561e+00, 5.35351131e+00,
1.23867704e+01],
[ 1.00000000e+00, -2.25644803e-01, 5.09155769e-02,
-1.14888353e-02],

```


[1.00000000e+00, 2.68474946e+00, 7.20787967e+00,
 1.93513510e+01],
 [1.00000000e+00, -2.54566474e-01, 6.48040897e-02,
 -1.64969486e-02],
 [1.00000000e+00, 2.49729116e+00, 6.23646316e+00,
 1.55742644e+01],
 [1.00000000e+00, 2.49424278e+00, 6.22124706e+00,
 1.55173006e+01],
 [1.00000000e+00, 2.56215366e-01, 6.56463136e-02,
 1.68195942e-02],
 [1.00000000e+00, 1.58397282e+00, 2.50896990e+00,
 3.97414013e+00],
 [1.00000000e+00, 2.00157594e+00, 4.00630626e+00,
 8.01892623e+00],
 [1.00000000e+00, -6.88691216e-01, 4.74295591e-01,
 -3.26643207e-01],
 [1.00000000e+00, 9.19413500e-01, 8.45321185e-01,
 7.77199709e-01],
 [1.00000000e+00, 2.36883889e+00, 5.61139768e+00,
 1.32924970e+01],
 [1.00000000e+00, -1.60921519e-01, 2.58957354e-02,
 -4.16718108e-03],
 [1.00000000e+00, 1.17964304e+00, 1.39155771e+00,
 1.64154137e+00],
 [1.00000000e+00, -1.06847378e+00, 1.14163621e+00,
 -1.21980836e+00],
 [1.00000000e+00, 1.32503642e+00, 1.75572152e+00,
 2.32639496e+00],
 [1.00000000e+00, -1.08045846e+00, 1.16739048e+00,
 -1.26131692e+00],
 [1.00000000e+00, 5.58434058e-01, 3.11848597e-01,
 1.74146877e-01],
 [1.00000000e+00, 2.13412911e-02, 4.55450706e-04,
 9.71990608e-06],
 [1.00000000e+00, 5.30000537e-01, 2.80900569e-01,
 1.48877452e-01],
 [1.00000000e+00, -2.53509234e+00, 6.42669316e+00,
 -1.62922606e+01],
 [1.00000000e+00, -9.92732464e-01, 9.85517745e-01,
 -9.78355459e-01],
 [1.00000000e+00, -1.13058641e+00, 1.27822562e+00,
 -1.44514451e+00],
 [1.00000000e+00, 2.83613681e+00, 8.04367202e+00,
 2.28129543e+01],
 [1.00000000e+00, -5.84090735e-01, 3.41161987e-01,
 -1.99269556e-01],
 [1.00000000e+00, -2.00148279e+00, 4.00593337e+00,

-8.01780670e+00],
 [1.00000000e+00, -5.11457562e-01, 2.61588838e-01,
 -1.33791589e-01],
 [1.00000000e+00, 2.77567735e+00, 7.70438473e+00,
 2.13848862e+01],
 [1.00000000e+00, -1.81225366e+00, 3.28426331e+00,
 -5.95191820e+00],
 [1.00000000e+00, -1.47745656e+00, 2.18287788e+00,
 -3.22510725e+00],
 [1.00000000e+00, -8.07791546e-02, 6.52527182e-03,
 -5.27105941e-04],
 [1.00000000e+00, 6.70968142e-01, 4.50198248e-01,
 3.02068682e-01],
 [1.00000000e+00, -1.47843990e+00, 2.18578455e+00,
 -3.23155109e+00],
 [1.00000000e+00, -5.49128128e-01, 3.01541701e-01,
 -1.65585030e-01],
 [1.00000000e+00, 6.17469931e-01, 3.81269116e-01,
 2.35422215e-01],
 [1.00000000e+00, -1.20556444e+00, 1.45338562e+00,
 -1.75215002e+00],
 [1.00000000e+00, -9.14697968e-01, 8.36672373e-01,
 -7.65302519e-01],
 [1.00000000e+00, 1.26339490e+00, 1.59616667e+00,
 2.01658883e+00],
 [1.00000000e+00, -6.32047960e-01, 3.99484624e-01,
 -2.52493442e-01],
 [1.00000000e+00, 2.37580691e+00, 5.64445848e+00,
 1.34101435e+01],
 [1.00000000e+00, 2.31306273e+00, 5.35025919e+00,
 1.23754851e+01],
 [1.00000000e+00, -2.14689898e+00, 4.60917522e+00,
 -9.89543355e+00],
 [1.00000000e+00, 8.08960800e-02, 6.54417576e-03,
 5.29398166e-04],
 [1.00000000e+00, -2.85342893e+00, 8.14205666e+00,
 -2.32327800e+01],
 [1.00000000e+00, -3.97188635e-01, 1.57758812e-01,
 -6.26600070e-02],
 [1.00000000e+00, -2.96563010e+00, 8.79496190e+00,
 -2.60826038e+01],
 [1.00000000e+00, -2.48839749e+00, 6.19212206e+00,
 -1.54084610e+01],
 [1.00000000e+00, -1.56851922e+00, 2.46025254e+00,
 -3.85895338e+00],
 [1.00000000e+00, 2.32572192e+00, 5.40898244e+00,
 1.25797890e+01],

```
[ 1.00000000e+00, -2.29038346e+00,  5.24585638e+00,
 -1.20150227e+01],
 [ 1.00000000e+00, -3.32739319e-02,  1.10715455e-03,
 -3.68393850e-05],
 [ 1.00000000e+00, -2.60814290e+00,  6.80240941e+00,
 -1.77416558e+01],
 [ 1.00000000e+00,  1.64856508e+00,  2.71776681e+00,
  4.48041544e+00]])
```

```
[20]: regression_3 = LinearRegression()
      regression_3.fit(X_train_poly, y_train)
      y_pred = regression_3.predict(X_test_poly)
      polynomial_score_3 = r2_score(y_test, y_pred)
      polynomial_score_3
```

```
[20]: 0.9792861264275933
```

```
[21]: print(f"r2_score with Polynomial Regression deg 1: {linear_score} \nr2_score with
      ↪with Polynomial Regression deg 2: {polynomial_score} \nr2_score with
      ↪Polynomial Regression deg 3: {polynomial_score_3}")
```

```
r2_score with Polynomial Regression deg 1: 0.7887808396222011
r2_score with Polynomial Regression deg 2: 0.9792573702598509
r2_score with Polynomial Regression deg 3: 0.9792861264275933
```

Here you can see that clearly on the non linear dataset the linear regression is not give the good accuracy and the polynomial regression with degree 2 and degree 3 give the good accuracy

0.0.5 Prediction on new data set

```
[22]: X_new = np.linspace(-3, 3, 200).reshape(200, 1)
      X_new_poly = poly.transform(X_new) # deg=3
```

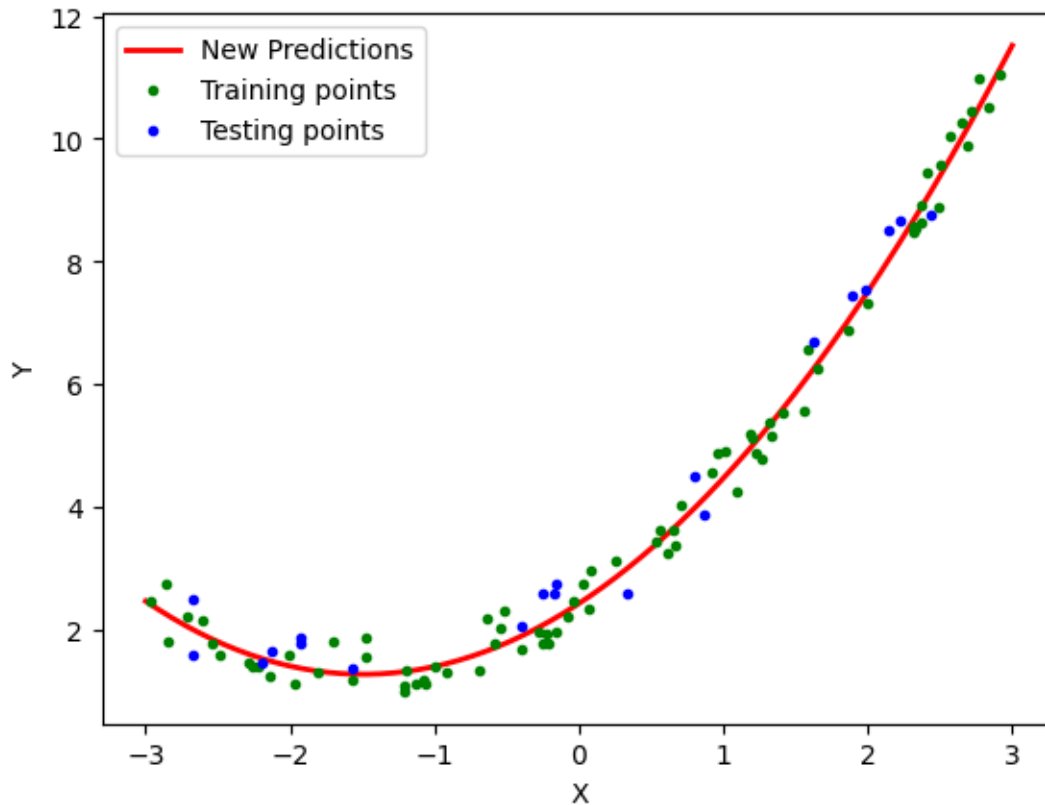
```
[23]: y_new = regression_3.predict(X_new_poly)
      plt.plot(X_new, y_new, 'r-', linewidth=2, label = "New Predictions")
      plt.plot(X_train, y_train, 'g.', linewidth=2, label = "Training points")
      plt.plot(X_test, y_test, 'b.', linewidth=2, label = "Testing points")
      plt.xlabel("X")
      plt.ylabel("Y")
      plt.legend()
      plt.show()

## Or

# y_new = regression_3.predict(X_new_poly)
# plt.plot(X_new, y_new, 'r-', linewidth=2, label="Predictions")
# plt.scatter(X_train, y_train, color='g', marker='.', linewidths=2,
↪label="Training points")
```

```
# plt.scatter(X_test, y_test, color='b', marker='.', linewidths=2,
#             label="Testing points")
# plt.xlabel("X")
# plt.ylabel("Y")
# plt.legend()
# plt.show()

## Both the code gives you same plot
```



0.0.6 Pipeline Concept

```
[24]: from sklearn.pipeline import Pipeline
```

```
[25]: def poly_regression(degree):
    X_new = np.linspace(-3, 3, 200).reshape(200, 1)
    poly_features = PolynomialFeatures(degree=degree, include_bias=True)
    lin_reg = LinearRegression()
    poly_regression = Pipeline([
        ("poly_features", poly_features),
        ("lin_reg", lin_reg)
```

```

])

poly_regression.fit(X_train, y_train)
y_pred_new = poly_regression.predict(X_new)

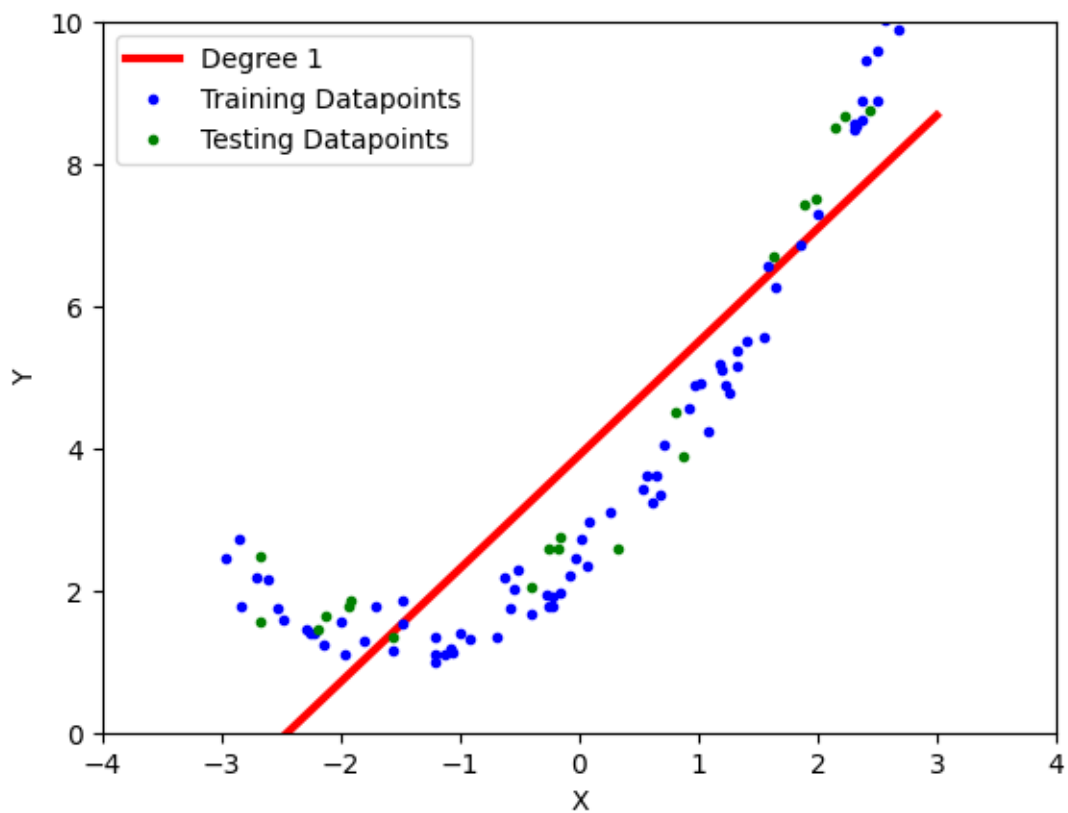
## Plotting the prediction line
plt.plot(X_new, y_pred_new, 'r', label = 'Degree ' + str(degree),
↳linewidth=3)
plt.plot(X_train, y_train, 'b.', linewidth=3, label = 'Training_
↳Datapoints')
plt.plot(X_test, y_test, 'g.', linewidth=3, label = 'Testing_
↳Datapoints')
plt.legend(loc= 'upper left')
plt.xlabel('X')
plt.ylabel('Y')
plt.axis([-4, 4, 0, 10])
plt.show()

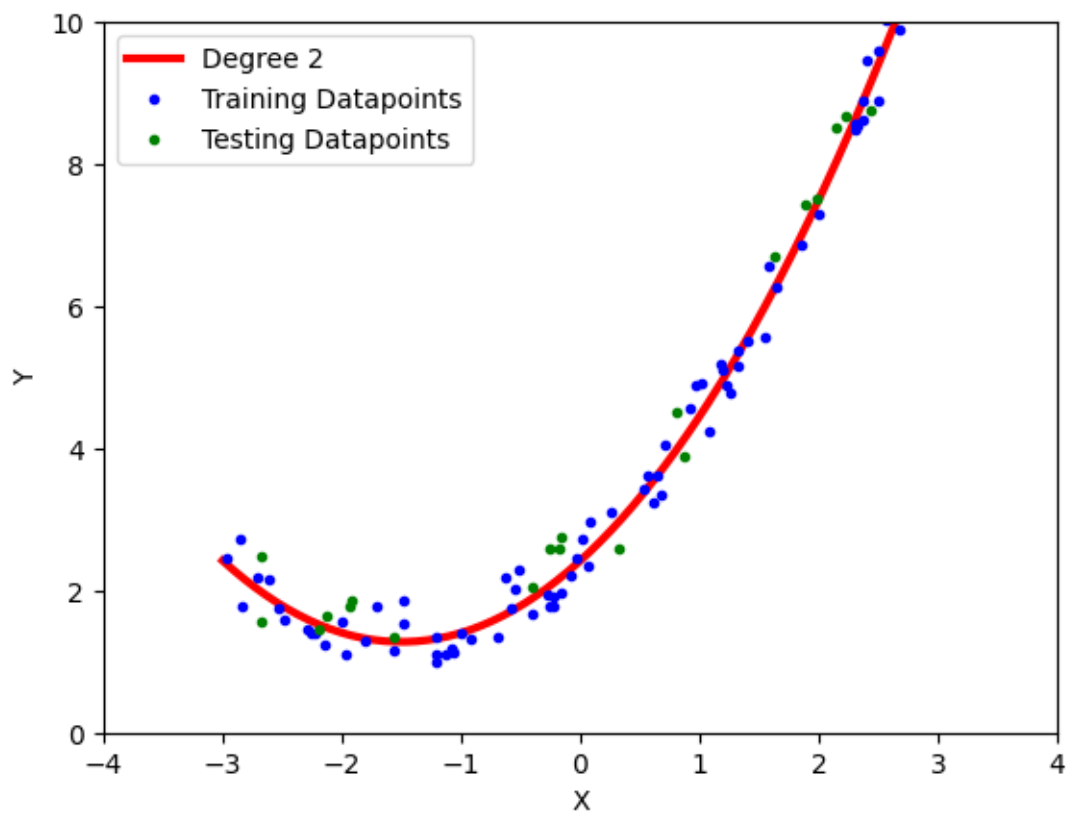
```

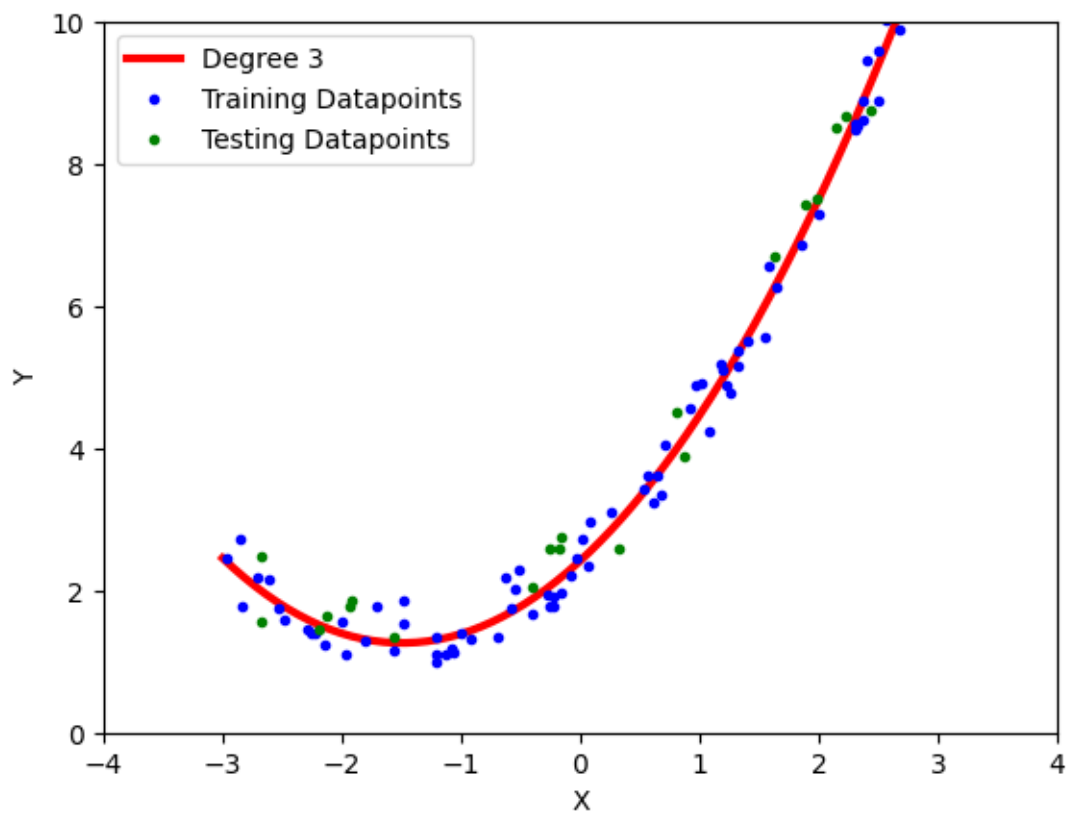
```

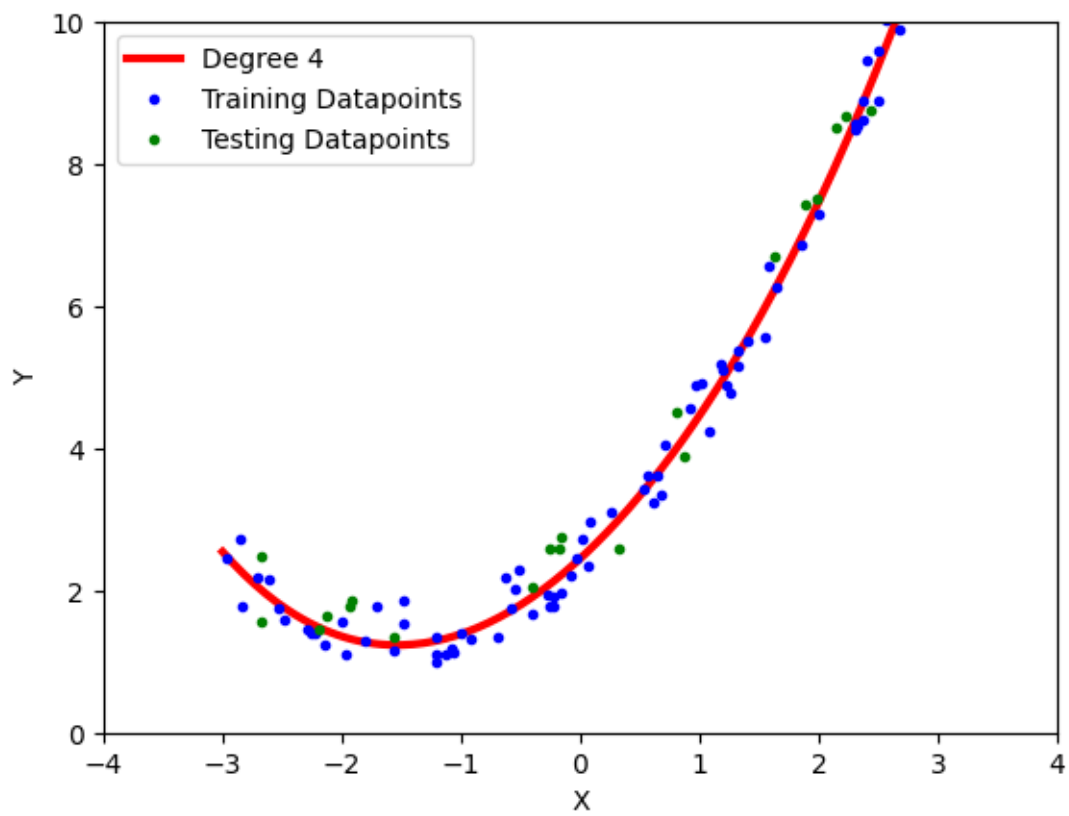
[26]: for i in range(1,7):
      poly_regression(i)

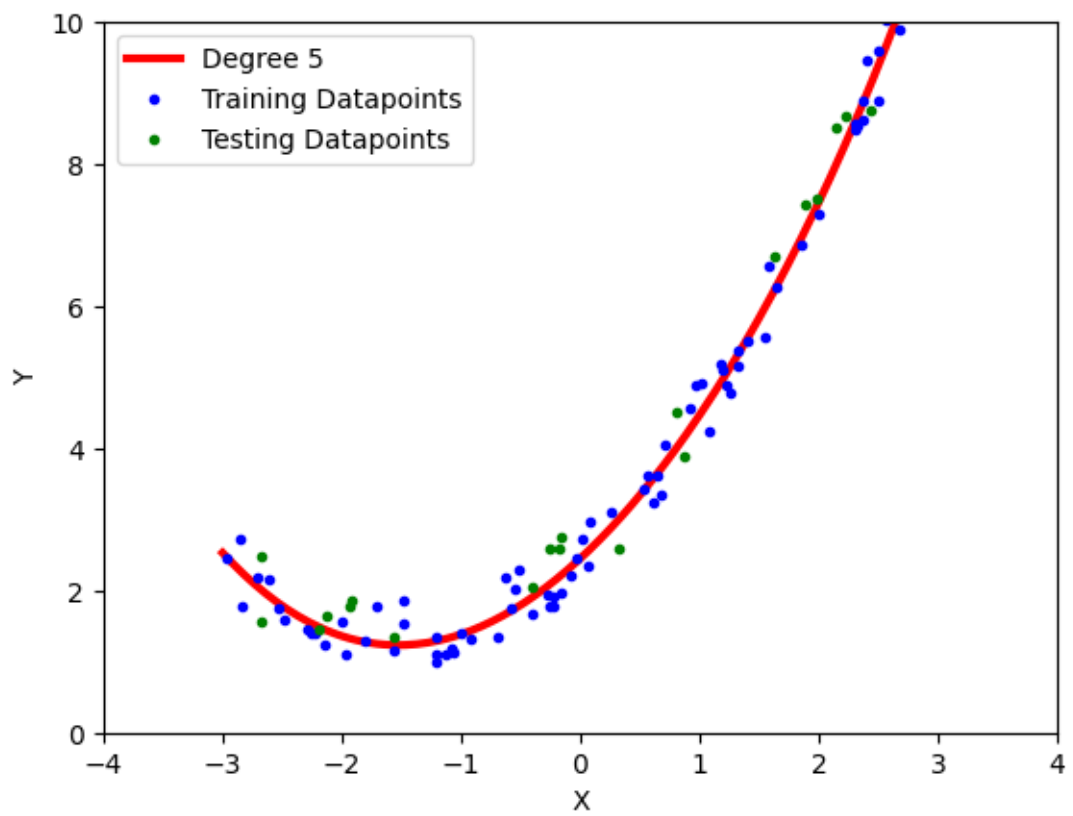
```

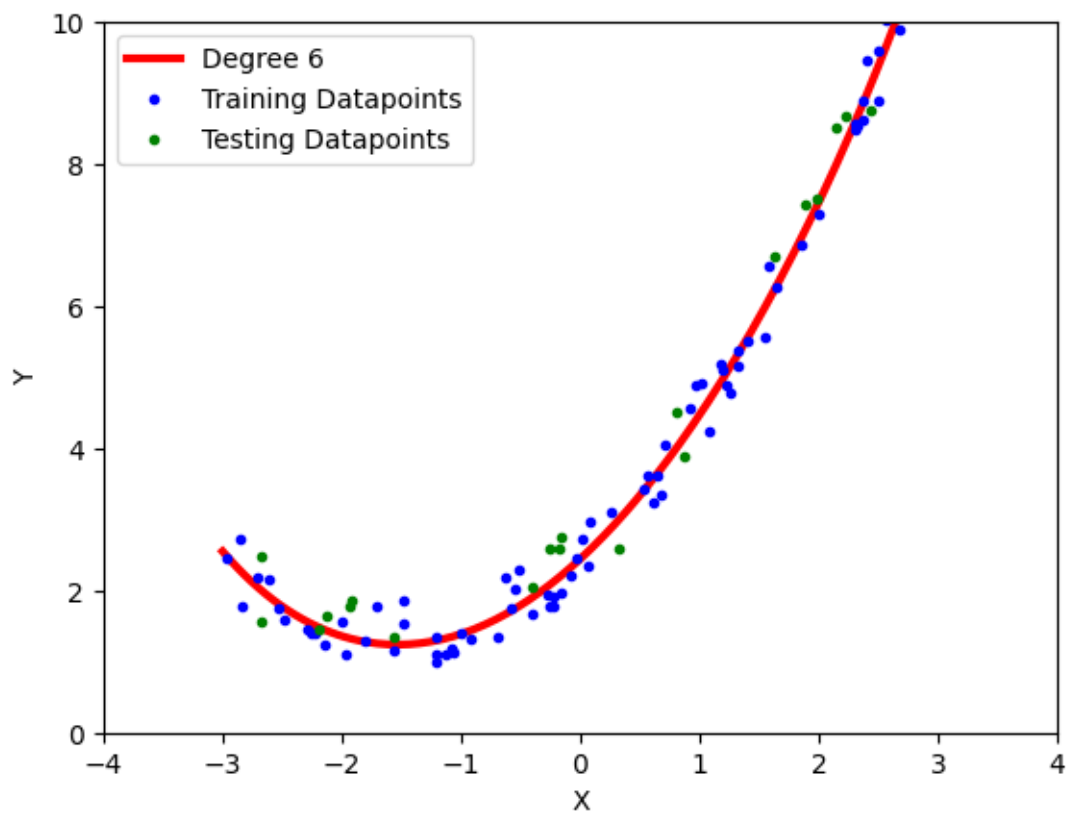




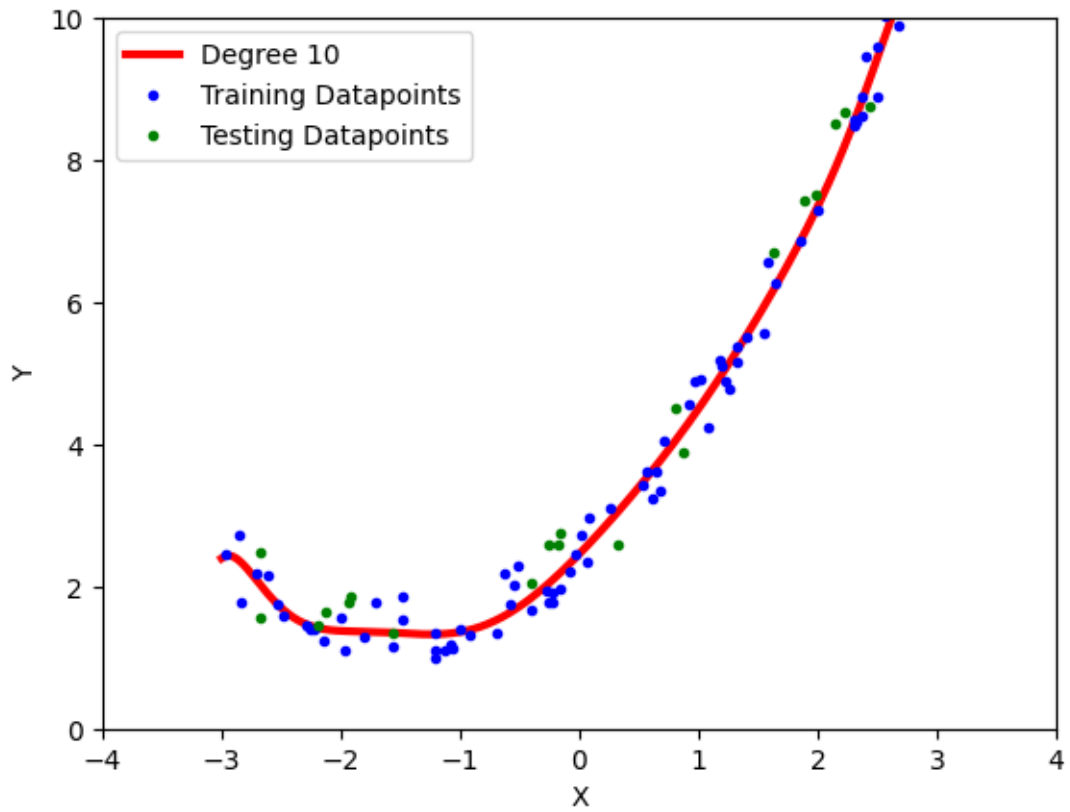








```
[27]: poly_regression(10) ## Overfitting the Model
```



```
[28]: poly_regression(15) ## Overfitting the Model
```

